UK Electricity Utility Assignment

# UK Wind Generation Forecast Accuracy

Decision-support and forecast-accuracy report for national Elexon wind data

| | |
|---|---|
| **Report date** | 25 October 2026 |
| **Current milestone** | 3 - WindLens React dashboard |
| **Application status** | Backend + WindLens frontend implemented locally |
| **API investigation** | Live Elexon requests succeeded |
| **Deployment** | Not deployed. No public URL exists. |
| **Data scope** | WINDFOR forecasts and FUELHH WIND actuals, January 2026 |

This printable report is generated from docs/PROJECT_REPORT.md. Milestone 1 verifies the Elexon data contracts and records the analytical rules. The React dashboard, FastAPI service, tests, Docker setup, and public deployment are planned and are not claimed as complete.

## Contents

1. Business problem and proposed solution
2. Why wind generation forecasting matters
3. Actual generation versus forecast generation
4. startTime and publishTime
5. Historical forecast publication times
6. Data sources and verified API behaviour
7-11. Pipeline, architecture, files, planned API and UI
12. Statistical formulas and worked example
13-15. Data quality, testing, Docker and deployment
16-18. Assumptions, customer demo, technical review
19. How to Explain This Project in an Interview

**Decision-support application for a UK electricity utility** Technical selection assignment Report date: 25 October 2026 Current milestone: **3 - WindLens React dashboard (implemented and tested)**

This report is the source of truth for the PDF. Update the Markdown first, then regenerate `docs/PROJECT_REPORT.pdf`.

## Feature status

| Area | Status | Notes |
|---|---|---|
| Workspace inspection | Implemented | Initial milestone |
| API investigation | Tested | Live Insights API requests succeeded on 25 October 2026 |
| Documentation and diagrams | Implemented | `docs/` |
| PDF generation | Implemented | `scripts/generate_report.py` |
| Elexon HTTP client | Implemented, tested | `backend/app/clients/elexon.py` — verified parameters only |
| Forecast selection and matching | Implemented, tested | Cutoff-aware latest publish; hourly actual from two half-hours |
| MAE / RMSE / bias | Implemented, tested | `backend/app/services/metrics.py` + synthetic Pytest |
| FastAPI backend | Implemented, tested | `/api/health`, `/api/config`, `/api/coverage`, `/api/metrics`, `/api/series` |
| January 2026 live pipeline | Tested | Integration test: 744 matched hourly pairs, 0 unmatched at default cutoff |
| Pytest suite | Tested | 21 unit/API tests + 2 integration tests passed on 25 October 2026 |
| Elexon source cache | Implemented, tested | TTL cache in `source_cache.py`; default 3600 s |
| Analytical correctness audit | Tested | Live Elexon reconciliation; see `docs/AUDIT_ANALYSIS.md` |
| WindLens React frontend | Implemented, tested | Vite + Recharts; live API via proxy; browser verified 25 Oct 2026 |
| Frontend unit tests | Tested | Vitest 3 passed (`frontend/src/utils/format.test.js`) |
| Docker and deployment artifacts | Implemented locally | `backend/Dockerfile`, `frontend/Dockerfile`, `docker-compose.yml`, `docs/DEPLOYMENT.md` |
| Public deployment | Not performed | No production URL in this repository |

### Status meanings

- Planned: designed, not built
- Implemented: present in this repository
- Tested: confirmed by a live check or automated test
- Deployed: available at a real public address

The WindLens dashboard is implemented under `frontend/` and consumes the live FastAPI backend.

## Executive summary

A UK electricity utility must buy energy from the wholesale day-ahead market before customers consume it. National wind generation is large and weather-dependent. The purchasing team therefore needs two things: the wind forecast that was available at decision time, and an honest measure of how accurate that forecast later proved.

This assignment is limited to national UK wind generation forecasts and actuals from Elexon for January 2026 delivery periods. No demand, price, or other-fuel dataset is included. The product is a decision-support and forecast-accuracy tool. It does not place trades and it does not compute the utility's complete buy volume.

Milestone 1 inspected the empty workspace, verified the Elexon APIs, and created the learning documentation. Live requests confirmed:

- 1,488 FUELHH WIND actuals covering every half-hour from 1 to 31 January 2026
- historical WINDFOR forecasts whose `startTime` values include January 2026 delivery periods
- a documented parameter set that must be respected, especially WINDFOR's publish-time-only filters

Milestone 2 delivered the analysis engine. Milestone 3 added **WindLens**, a Vite + React dashboard that calls `/api/metrics` and `/api/series`, shows KPI cards and Recharts visualisations, and exposes the decision cutoff control. Screenshot: `docs/screenshots/windlens-dashboard.png`. Next: Docker Compose and public deployment.

*Figure 1. Business workflow from Elexon data to decision support.*

## 1. Business problem and proposed solution

### Simple explanation

The company sells electricity to homes and businesses. It must buy that electricity before people use it. A lot of UK electricity comes from wind. If the wind forecast is wrong, the company may buy too much or too little from the day-ahead market and then have to correct the mistake later.

### Technical explanation

The application will reconstruct, for each January 2026 delivery period, the latest national wind forecast that would have been available at a configurable decision cutoff. It will compare that forecast with the corresponding FUELHH WIND actual and summarise the errors.

### Example

For delivery starting `2026-01-01T00:00:00Z`, the default cutoff is 09:20 UK time on 31 December 2025. The selected forecast is the newest WINDFOR row for that `startTime` with `publishTime` at or before the cutoff. That value is then compared with the actual wind outturn.

### Interview-ready definition

This product is a hindcast decision-support tool: it evaluates the forecast a buyer could have used, not the best forecast visible with hindsight.

### What the product will not do

- automatically buy electricity

- forecast customer demand
- include gas, nuclear, solar, interconnector, or other fuels in the accuracy metrics
- use prices
- claim an official National Energy System Operator or Elexon purchasing cutoff

## 2. Why wind generation forecasting matters

### Simple explanation

Wind is free fuel, but it is not reliable in the short term. A buyer who ignores the wind forecast is guessing.

### Technical explanation

National wind output in the January 2026 FUELHH sample ranged from 1,695 MW to 17,805 MW. That swing is large enough to change how much other power a supplier may need to buy or sell. Forecast accuracy therefore affects imbalance risk even though this app does not calculate the full stack.

### Example

If the forecast says 15,000 MW of wind and the actual is 8,000 MW, the system is short of wind by 7,000 MW. A supplier that leaned on that forecast may need more wholesale power.

### Interview-ready definition

Wind forecast accuracy matters because wind is a high-volume, weather-driven, low-marginal-cost source whose error feeds directly into day-ahead procurement risk.

## 3. Actual generation versus forecast generation

### Simple explanation

The actual is what happened. The forecast is what someone predicted before it happened.

### Technical explanation

- **Actuals** come from Elexon dataset FUELHH, filtered to `fuelType = WIND`. `generation` is the half-hourly average MW outturn. `startTime` is the delivery-period start.
- **Forecasts** come from Elexon dataset WINDFOR. `generation` is the forecast MW. `startTime` is the delivery period being forecast. `publishTime` is when that forecast version was issued.

### Example

Verified 1 January 2026, settlement period 1:

- actual: `startTime = 2026-01-01T00:00:00Z`, `generation = 13756`
- one earlier forecast: `publishTime = 2025-12-31T04:30:00Z`, `startTime = 2026-01-01T00:00:00Z`, `generation = 19556`

Those two numbers are not yet a formal accuracy result. They are only an illustration that both records exist.

### Interview-ready definition

Actual generation is measured outturn. Forecast generation is a published expectation for a future delivery period, versioned by `publishTime`.

## 4. The meaning of startTime and publishTime

### Simple explanation

`startTime` answers "which half-hour or hour are we talking about?" `publishTime` answers "when did this number appear?"

### Technical explanation

For both datasets, inspected timestamps were ISO-8601 UTC values ending in `Z`.

For FUELHH, `startTime` is the start of a 30-minute settlement period. `publishTime` is when that outturn was published and is not used as the delivery key.

For WINDFOR, `startTime` is the delivery time being forecast. `publishTime` identifies the forecast version. Many rows can share one `startTime` and differ only by `publishTime`.

### Example

Two WINDFOR rows can both forecast `2026-01-01T00:00:00Z`. One published at `2025-12-31T02:30:00Z` with 19,512 MW. Another published at `2025-12-31T04:30:00Z` with 19,556 MW. Same delivery period, different information sets.

### Interview-ready definition

`startTime` is the delivery-period identifier. `publishTime` is the information-vintage identifier.

## 5. Why historical forecast publication times matter

### Simple explanation

A later forecast is often better because the weather models had more time. Using today's latest forecast to judge yesterday's purchase is cheating.

### Technical explanation

The unfiltered WINDFOR stream returns only the latest published forecast. That is the wrong object for historical decision analysis. Dataset endpoints exist specifically so superseded forecasts can be retrieved by publish-time filters.

Three ideas must stay separate:

- latest forecast available at a historical decision cutoff
- latest forecast currently stored in the historical dataset
- actual generation for the delivery period

A forecast published after the cutoff, and especially a forecast published after delivery, must not be used to score a historical purchasing decision.

### Example

If the cutoff is 09:20 on D-1, a 11:30 publish on D-1 is ineligible even if it is closer to the actual.

### Interview-ready definition

Historical publication times allow as-of analysis: accuracy conditional on the information set that existed at decision time.

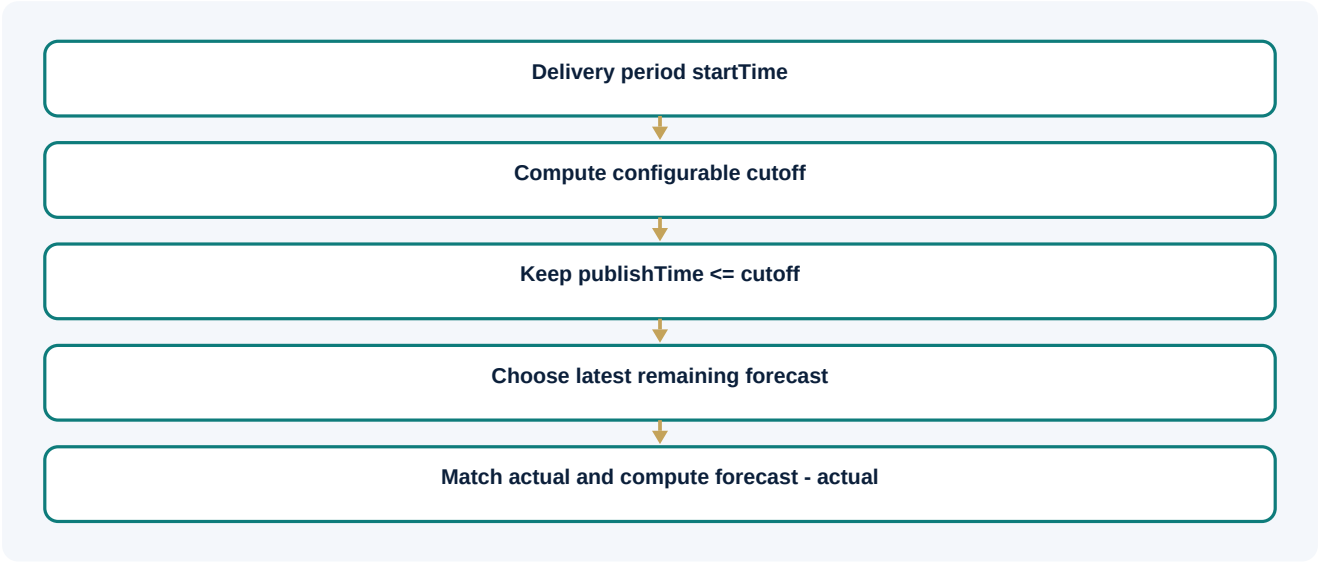


*Figure 3. As-of forecast selection for one delivery period.*

## 6. Data sources and verified API behaviour

Full notes: `docs/investigation/API_FINDINGS.md` Probe output: `docs/investigation/elexon_api_probe.json`

### Access

| Item | Finding | Confidence |
|---|---|---|
| Base URL | `https://data.elexon.co.uk/bmrs/api/v1` | Verified |
| API key | Not required | Verified by successful unauthenticated GETs |
| Rate limit | 5,000 requests/minute documented | Documented, not load-tested |
| Stream paging | No page token documented or observed | Verified for the January FUELHH extract |

### FUELHH stream

`GET /datasets/FUELHH/stream`

Verified filters: `settlementDateFrom`, `settlementDateTo`, `fuelType`. Documented extra filters: `settlementPeriod`, `publishDateTimeFrom`, `publishDateTimeTo`. Restriction, verified by HTTP 400: settlement-date and publish-date filters cannot be combined.

January 2026 WIND extract:

- 1,488 rows
- 31 settlement dates
- settlement periods 1-48 complete on every date
- no missing `generation`
- values from 1,695 MW to 17,805 MW
- half-hourly `startTime` values
- response order is not chronological; sort in code

### WINDFOR stream

`GET /datasets/WINDFOR/stream`

Verified filters: none (latest forecast only), or `publishDateTimeFrom` / `publishDateTimeTo` for history.

No documented delivery-period filter. An undocumented `startTime` query parameter was ignored.

Observed fields: `dataset`, `publishTime`, `startTime`, `generation`.

Historical availability:

- 31 December 2025 had 8 publishes and 584 rows, including 368 January 2026 start times
- early 30 January 2026 publishes included start times through 1 February 2026

Observed publish times on 31 December 2025 were 02:30, 04:30, 07:30, 09:30, 11:30, 15:30, 18:30 and 22:30 UTC. Elexon documents 03:30, 05:30, 08:30, 10:30, 12:30, 16:30, 19:30 and 23:30. The UTC field in the data is the selection source of truth.

### Units

FUELHH official text: average MW over 30 minutes. WINDFOR official wind page and related docs: MW. OpenAPI types: integer `generation` values, nullable.

### Planned retrieval window

Because WINDFOR cannot be filtered by delivery date on the server:

- request publishes from `2025-12-25T00:00:00Z` to `2026-01-31T23:59:59Z`
- keep rows whose `startTime` is a January 2026 delivery period
- apply the cutoff in code

The full WINDFOR archive was not downloaded in this milestone.

### Fallback if live access fails later

- record the failure
- store a real filtered snapshot under `data/raw/` with provenance
- read that snapshot through configuration
- never invent Elexon observations

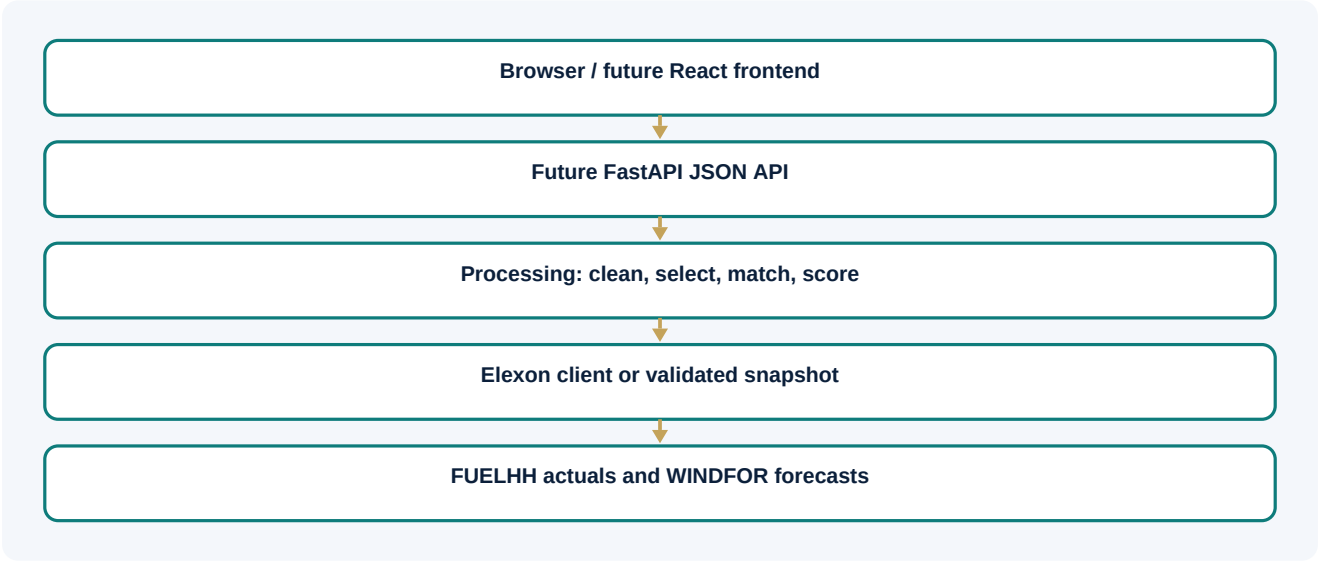


*Figure 2. Planned system architecture. Only documentation exists today.*

## 7. Data processing pipeline

Status: **implemented** in `backend/app/services/` and orchestrated by `backend/app/services/analysis.py`.

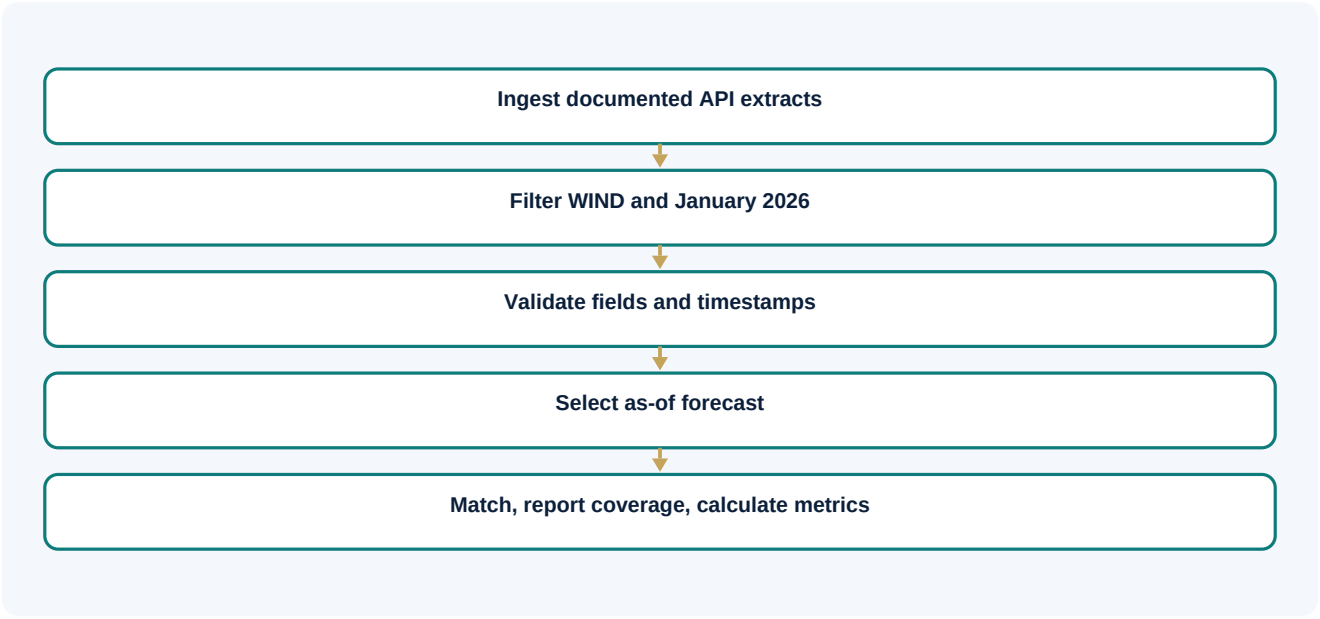


*Figure 4. Planned data-processing pipeline.*

- **Ingest** FUELHH and WINDFOR through documented parameters, or load a validated snapshot.
- **Filter** FUELHH to `fuelType = WIND` and January 2026 settlement dates. Filter WINDFOR to January 2026 `startTime` values.
- **Validate** required fields, types, and non-null generation.
- **Normalise timestamps** to timezone-aware UTC.
- **Select** the latest eligible forecast for each delivery period.
- **Match** forecasts to actuals using the resolution rule below.
- **Report coverage** before metrics.
- **Calculate** MAE, RMSE, and bias on valid pairs only.
- **Serve** JSON to the future dashboard.

### Planned matching rule for mixed resolution

Verified fact: WINDFOR `startTime` values are hourly. FUELHH `startTime` values are half-hourly.

Planned default, labelled as an analytical assumption:

- treat each WINDFOR row as a forecast for the hour beginning at `startTime`
- average the two FUELHH WIND half-hours that start at `HH:00` and `HH:30`
- compare forecast MW with that hourly mean actual MW

If one of the two half-hours is missing, that hour is unmatched. It is not filled with zero.

An alternative exact-`startTime` join would use only the `:00` actual and discard half the outturn. That alternative will be documented in code comments if implemented, but it is not the intended default.

## 8. Application architecture

Status: **backend implemented**; frontend still planned.

Layers:

- React + Vite frontend
- FastAPI backend

- Pandas processing services
- Elexon client or snapshot reader

Deliberately excluded: database, authentication, machine learning, message queues.

Configuration will use environment variables. See `.env.example`. No secrets are required for the public Insights API.

## 9. Purpose of every major directory and file

| Path | Purpose | Status |
|---|---|---|
| `README.md` | How to orient yourself in the repo | Implemented |
| `.env.example` | Public configuration template | Implemented |
| `requirements-docs.txt` | ReportLab dependency for PDF generation | Implemented |
| `docs/PROJECT_REPORT.md` | This report | Implemented |
| `docs/PROJECT_REPORT.pdf` | Printable report | Implemented after generation |
| `docs/LEARNING_GUIDE.md` | Beginner teaching notes | Implemented |
| `docs/PROGRESS.md` | Honest checklist | Implemented |
| `docs/diagrams/` | Mermaid diagrams and explanations | Implemented |
| `docs/investigation/` | Verified API notes and probe JSON | Implemented |
| `scripts/investigate_elexon_api.py` | Reproducible live probe | Tested |
| `scripts/generate_report.py` | Markdown-to-PDF generator | Implemented |
| `backend/app/main.py` | FastAPI routes | Implemented |
| `backend/app/clients/elexon.py` | Live Elexon client | Implemented |
| `backend/app/services/*.py` | Selection, matching, metrics | Implemented |
| `tests/test_*.py` | Pytest (synthetic + integration) | Tested |
| `frontend/` | Future React app | Planned placeholder |
| `data/` | Future snapshots and processed tables | Empty by design |
| `tests/` | Future Pytest suite | Planned placeholder |

## 10. Backend API endpoints

Status: **implemented**. Run with:

`.\scripts\start-backend.ps1` (default **127.0.0.1:8001**; see `dev.env.example`)

| Route | Purpose |
|---|---|
| `GET /api/health` | Service liveness |
| `GET /api/config` | Delivery window, cutoff, Elexon base URL |
| `GET /api/coverage` | Half-hour count, hourly actuals, matched/unmatched hours |
| `GET /api/metrics` | MAE, RMSE, bias, coverage (`?cutoff=HH:MM` optional override) |
| `GET /api/series` | Matched hourly rows (`?limit=` max 2000, optional `?cutoff=HH:MM`) |

### Verified JSON shapes

`GET /api/metrics` response:

```
{
  "metrics": { "n": 744, "mae": 2142.12, "rmse": 2611.70, "bias": 1851.90 },
  "coverage": {
    "actual_half_hours_in_window": 1488,
    "hourly_actuals_with_both_half_hours": 744,
    "forecast_rows_january_delivery": 18080,
    "matched_hourly_pairs": 744,
    "unmatched_hours": 0,
    "metrics_sample_size": 744
  },
  "cutoff_used": "09:20"
}
```


`GET /api/series` each element of `series`:


```
{
  "hour_start": "2026-01-01T00:00:00Z",
  "settlement_date": "2026-01-01",
  "forecast_mw": 19556.0,
  "actual_mw": 13756.0,
  "forecast_error": 5800.0,
  "publish_time": "2025-12-31T04:30:00Z",
  "cutoff_utc": "2025-12-31T09:20:00Z"
}
```


Raw FUELHH and WINDFOR payloads are cached in-process (`ELEXON_SOURCE_CACHE_TTL_SECONDS`, default **3600**). `/api/metrics`, `/api/coverage`, and `/api/series` reuse the same source rows within TTL; cutoff-specific selection and metrics still run on each request. Set TTL **0** to fetch Elexon on every request. See `backend/README.md`.

## 11. WindLens frontend (React)

Status: **implemented and browser-tested** on 25 October 2026.

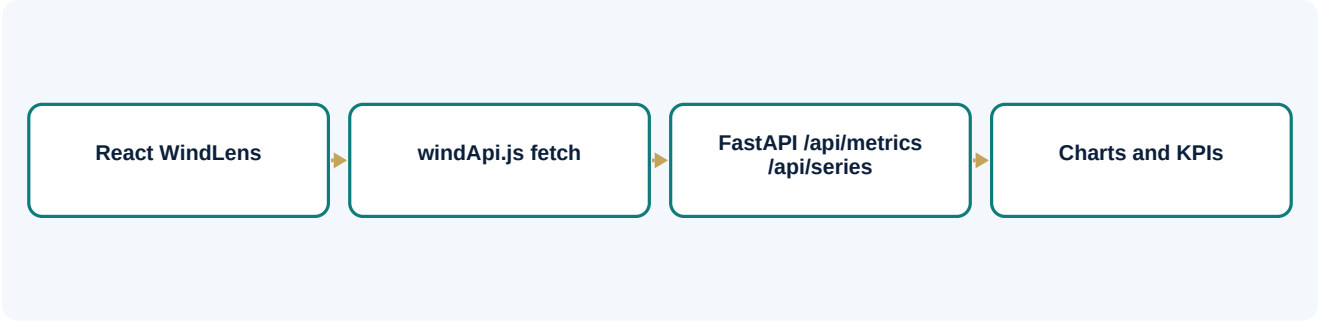


*Figure 6. WindLens frontend data flow (implemented).*

Product name in the UI: **WindLens** with subtitle **UK Wind Forecast Performance**. Compact header metadata: January 2026, historical analysis, Source: Elexon.

Navigation (in-app sections, no dead links): **Overview**, **Forecast Analysis**, **Methodology**.

| File / component | Role |
|---|---|
| `frontend/src/App.jsx` | Sidebar navigation, shared cutoff/date state, `useAnalysis` wiring |
| `frontend/src/pages/OverviewPage.jsx` | Key insight banner, four KPIs, main forecast vs actual chart, analysis settings |
| `frontend/src/pages/ForecastAnalysisPage.jsx` | Error chart + paginated delivery-period table |
| `frontend/src/pages/MethodologyPage.jsx` | Sources, cutoff assumption, metrics definitions, limitations |
| `frontend/src/hooks/useAnalysis.js` | Loads config, metrics, series when cutoff changes |
| `frontend/src/api/windApi.js` | Single HTTP module for all `/api/*` calls |
| `frontend/src/components/CompactHeader.jsx` | Title, subtitle, compact historical/Elexon metadata |

| File / component | Role |
|---|---|
| `frontend/src/components/InsightBanner.jsx` | Plain-English bias insight from API metrics |
| `frontend/src/components/KpiGrid.jsx` | MAE, bias, RMSE, matched hours / coverage (four cards) |
| `frontend/src/components/AnalysisSettings.jsx` | Cutoff time, presets, Recalculate → `?cutoff=` |
| `frontend/src/components/ForecastChart.jsx` | Recharts forecast (blue) vs actual (teal) |
| `frontend/src/components/ErrorChart.jsx` | Signed forecast error bars with zero line |
| `frontend/src/components/PeriodTable.jsx` | Sortable/searchable table with pagination |
| `frontend/src/components/StatusBanner.jsx` | Loading spinner, error + Retry |

Configuration: `VITE_API_BASE_URL` (empty in dev → Vite proxy; default backend **8001**). See `frontend/.env.example` and `frontend/.env.development`.

**Not implemented:** forecast evolution across all WINDFOR vintages (API returns only the selected as-of row per hour).

Screenshot (actual running app): `docs/screenshots/windlens-dashboard.png`.

## 11.1 Analytical correctness audit (25 October 2026)

Status: **completed** — no pipeline defects found; dashboard headline metrics match independently recomputed values from the matched series.

A full trace, unit verification, and live reconciliation are documented in `docs/AUDIT_ANALYSIS.md`. Verified on live Elexon data with default cutoff **09:20 Europe/London**:

| Check | Result |
|---|---|
| FUELHH half-hours (Jan 2026 WIND) | 1,488 |
| Complete hourly actuals | 744 |
| Matched forecast–actual pairs | 744; 0 unmatched |
| `publish_time <= cutoff_utc` | 0 violations |
| Re-run `select_latest_eligible_forecast` | 0 mismatches |
| MAE / RMSE / bias vs independent calc | Exact match (744 rows) |

**Verified facts:** Error sign convention, metric formulas, cutoff timezone handling for January (GMT), and exclusion of hours missing either half-hour actual.

**Assumptions (not schema-proven):** WINDFOR `generation` applies to the hour starting at `startTime`; January delivery filtering by UTC `start_time` aligns with UK settlement dates for this month (0 mismatches observed).

**Uncorrected limitations:** WINDFOR fetched by publish window only (client-side January `startTime` filter); no forecast-evolution history; `/api/series` truncates with `limit` (frontend uses 2000).

Reconciliation script: `scripts/audit_analysis.py`.

## 12. Statistical formulas

Status: **implemented and tested** in `backend/app/services/metrics.py`. The worked example below is **synthetic** and is not Elexon data.

Convention:

```
e_i = forecast_i - actual_i
```

Only valid matched observations are used. `n` is the number of those observations. If `n = 0`, metrics are undefined and must be reported as unavailable, not as zero.

### Mean Absolute Error

```
MAE = (1 / n) * sum(|e_i|)
```

Simple: the average size of the mistake. Technical: L1 mean error in MW. Interview: MAE is the average absolute forecast error and is robust to sign but not to the need for a typical magnitude.

### Root Mean Squared Error

```
RMSE = sqrt((1 / n) * sum(e_i^2))
```

Simple: like MAE, but big mistakes hurt more. Technical: L2 mean error, same units as generation. Interview: RMSE penalises large errors more than MAE and is always at least as large as MAE.

### Mean forecast error / bias

```
Bias = (1 / n) * sum(e_i)
```

Simple: the average signed mistake. Technical: `E[forecast - actual]`. Interpretation required by this assignment:

- positive bias: forecasts tend to overestimate wind generation
- negative bias: forecasts tend to underestimate wind generation

### Worked synthetic example

| Period | Forecast MW | Actual MW | Error |
|---|---|---|---|
| A | 100 | 90 | 10 |
| B | 80 | 100 | -20 |

`MAE = (10 + 20) / 2 = 15` `RMSE = sqrt((100 + 400) / 2) = sqrt(250) = 15.81` `Bias = (10 + -20) / 2 = -5`

The pair is negatively biased: it underestimates on average.

## 13. Data quality checks

### Already observed in the investigation

- January 2026 FUELHH WIND coverage is complete at 1,488 half-hours
- no null FUELHH generation in that extract
- WINDFOR historical rows exist for early and late January delivery times
- no null generation in the probed WINDFOR windows
- WINDFOR/FUELHH time resolution differs
- FUELHH result order is reverse-chronological

### Planned runtime checks

- required fields present
- timestamps parse as UTC
- `publishTime <= startTime` is not required for WINDFOR because some rows forecast periods already started; however, a forecast with `publishTime` after the cutoff is ineligible
- duplicate `(startTime, publishTime)` rows are detected
- missing actuals are counted, not zero-filled

- metrics carry an `n` and a coverage percentage

## 14. Testing and validation

Status: **tested** on 25 October 2026.

Commands:

```
python -m pip install -r requirements.txt
python -m pytest -m "not integration"
python -m pytest -m integration
```

Results:

- **9 passed** unit/API tests (synthetic data only in unit tests)
- **1 passed** integration test against live Elexon for January 2026

Integration run at default cutoff `09:20 Europe/London`:

| Check | Result |
|---|---|
| FUELHH half-hours in window | 1,488 |
| Complete hourly actuals | 744 |
| WINDFOR rows with January `startTime` | 18,080 (after client-side filter) |
| Matched hourly pairs | 744 |
| Unmatched hours | 0 |
| MAE | ~2,142 MW |
| RMSE | ~2,612 MW |
| Bias | ~+1,852 MW (forecasts tended to overestimate wind) |

Positive bias matches the assignment convention: forecast minus actual greater than zero means overestimation.

Unit tests cover cutoff time, pre/post-cutoff selection, synthetic MAE/RMSE/bias, hourly aggregation, and mocked API responses.

## 15. Docker and deployment

Status: **artifacts implemented**; **no public URL** has been created in this repository.

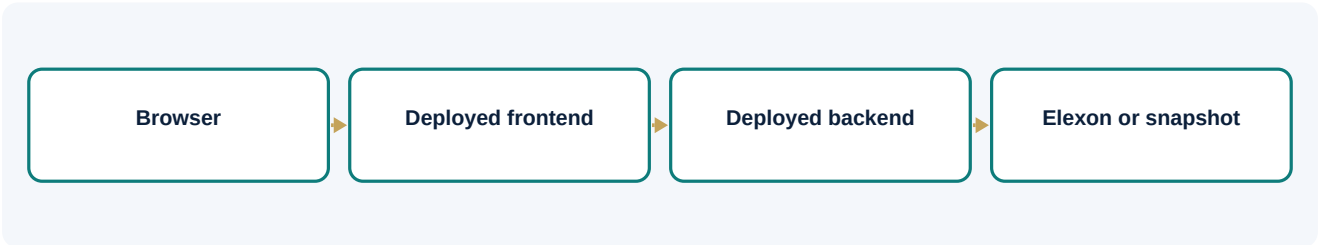


*Figure 5. Planned deployment path. No public URL exists yet.*

- `backend/Dockerfile` — FastAPI on `0.0.0.0:8001`, env-driven settings, outbound Elexon HTTPS.
- `frontend/Dockerfile` — Vite production build + nginx serving static files.
- `deploy/nginx.conf` — SPA routing and `/api` reverse proxy to the backend service.
- `docker-compose.yml` — local stack on **http://localhost:8080** (verified running 25 Oct 2026; not deployed to the internet).
- `docs/DEPLOYMENT.md` — hosting options, CORS, `VITE_API_BASE_URL`, cache and storage notes.

Public deployment requires you to push images or static assets to a chosen host and set environment variables. See `docs/DEPLOYMENT.md`.

## 16. Important assumptions and limitations

- **Decision cutoff default:** 09:20 Europe/London on the calendar day before the delivery period's UK settlement date. Rationale: public EPEX SPOT GB Day-Ahead 60-minute auction order-book close. This is an analytical assumption pending the client's actual process. It is configurable.
- **WINDFOR hour beginning:** `startTime` is treated as the start of the forecast hour. Official schema text does not prove this beyond field naming and FUELHH analogy.
- **Hourly matching:** two FUELHH half-hours will be averaged to compare with one WINDFOR hour.
- **Scope:** national WIND only, January 2026 only, Elexon only.
- **WINDFOR coverage:** NESO describes the forecast as covering ESO-visible wind farms with operational metering. FUELHH WIND is GB fuel-type outturn. They are the official pairing used on Elexon's own wind page, but they may not represent every embedded wind site.
- **No DST in the target month:** January 2026 has 48 settlement periods every day. Code must still be DST-safe.
- **Public deployment:** Docker Compose and deployment docs exist locally; no production URL is claimed unless you deploy.
- **Observed versus documented publish clocks:** use data timestamps, not the advertised 03:30-23:30 list, for selection.

## 17. How to demonstrate the application to a customer

- Terminal 1 (PowerShell, repo root): `.\scripts\start-backend.ps1` → http://127.0.0.1:8001/docs
- Terminal 2: `.\scripts\start-frontend.ps1` → open the Vite URL (5173 or 5174). Proxy: `frontend/.env.development`
- Show KPI cards loading from `/api/metrics` (not hardcoded).
- Scroll the forecast vs actual chart and error chart; open the delivery table.
- Change cutoff to `15:30` and Recalculate — MAE changes (verified: ~1981 MW vs ~2142 MW at 09:20).
- Explain that `08:00` and `09:20` may match when no publishes fall between them.

Use `docs/screenshots/windlens-dashboard.png` in slides if live demo is unavailable.

## 18. How to explain the implementation to a technical reviewer

- Start with scope and data contracts, not React components.
- Show verified parameters and the 400 error for illegal FUELHH filter combinations.
- Explain why WINDFOR must be pulled by publish window and filtered by `startTime` locally.
- Walk the as-of selection algorithm and the three-way distinction between cutoff-latest, dataset-latest, and actual.
- Explain hourly-to-half-hourly matching and why zeros are forbidden.
- Point at the future unit tests as the correctness mechanism.
- Keep the architecture boring on purpose: FastAPI, Pandas, no database.

## 19. How to Explain This Project in an Interview

**One-minute version**

"A UK supplier needs to buy power day-ahead. Wind is a big, weather-driven part of national generation. I am building a small React and FastAPI tool that reconstructs the latest Elexon wind forecast available at a historical decision time and scores it against January 2026 actuals using MAE, RMSE, and bias. The first milestone verified that the Elexon data exists and designed the selection rules before writing the app."

**Expected follow-up answers**

- Why not use the latest forecast? Because that is hindsight.
- What is bias? The average signed error. Positive means over-forecasting wind.
- What if an actual is missing? Drop the pair and report coverage.
- What is the cutoff? A configurable assumption, defaulting to 09:20 UK time on D-1.
- What did you verify? Complete January FUELHH WIND actuals and historical WINDFOR publishes that already contain January start times.

**What not to claim**

Do not claim public deployment. Do say WindLens, the FastAPI backend, Pytest, Vitest, and `npm run build` were verified on 25 October 2026.

## 20. How to generate and open the PDF

From the ELECTRICITY workspace:

```
python -m pip install -r requirements-docs.txt
python scripts/generate_report.py
```

The script reads this Markdown file, draws the architecture diagrams, and writes:

`docs/PROJECT_REPORT.pdf`

On Windows, open that file in any PDF reader. The generation command is the reproducibility mechanism. Do not hand-edit the PDF.